

Classification de sites web basée sur l'analyse de robots.txt

BARRY Souleymane

souleymane.barry@etu.unicaen.fr

MAFILLE Léo

leo.mafille@etu.unicaen.fr

Abstract

La classification automatique des pages web est une tâche fondamentale pour la recherche d'information, essentielle à des applications allant du filtrage parental au ciblage publicitaire. Les approches traditionnelles reposent principalement sur l'analyse sémantique du contenu visible (HTML) ou le rendu visuel. Bien qu'efficaces, ces méthodes sont coûteuses en ressources de calcul et en bande passante. Dans cet article, nous proposons une méthodologie de classification inédite et légère, basée uniquement sur le fichier robots.txt. Nous avons constitué un dataset basé sur le dataset Website classification using URL par Sayad Ahmed (Ahmed, 2023) et expérimenté avec un modèle d'apprentissage. Nos résultats démontrent que l'analyse du robots.txt n'est pas suffisante pour une classification efficace.

1 Introduction

La croissance exponentielle du World Wide Web rend la catégorisation manuelle des sites impossible. Les Fournisseurs d'Accès Internet (FAI), les agences de cybersécurité et les acteurs du marketing numérique ont un besoin critique d'outils capables de classer automatiquement des millions de domaines.

Historiquement, la classification de sites web a été traitée comme un problème de classification de texte (NLP). La méthode standard consiste à télécharger le code HTML complet de la page d'accueil, nettoyer le DOM (Document Object Model) et appliquer des algorithmes d'analyse sur le texte visible. Bien que les approches basées sur le contenu ("Content-based") offrent une grande précision sémantique, elles souffrent de limitations majeures en termes de scalabilité :

1. **Intensité des ressources** : Le téléchargement et le parsing de pages HTML complètes consomment une bande passante et une puissance de calcul significatives.

2. **Contraintes légales et éthiques** : Le scraping profond ("Deep packet inspection") ou l'analyse de contenus protégés peut soulever des problèmes de confidentialité.
3. **Web Dynamique** : Les sites modernes reposent massivement sur le rendu côté client (JavaScript), rendant l'analyse statique du HTML de plus en plus difficile sans l'usage de navigateurs "headless" lourds.

Pour pallier ces limitations, nous explorons une approche "basée sur la structure" ("Structure-based") utilisant le Protocole d'Exclusion des Robots. Le fichier robots.txt est un standard public, léger et strictement structuré.

Notre hypothèse centrale est que les chemins de répertoires et les règles définies dans le robots.txt trahissent souvent la pile technologique utilisée (par exemple, /wp-admin/ pour WordPress, /checkout/ pour Shopify). Puisque certaines technologies sont fortement corrélées à des catégories de sites spécifiques (Shopify pour le E-commerce, Moodle pour l'Éducation), le fichier robots.txt peut servir de proxy efficace pour la classification thématique.

Dans cet article, nous présentons :

- Une méthodologie pour construire un dataset en transformant le dataset Website classification using URL. (Ahmed, 2023)
- Un pipeline d'ingénierie des fonctionnalités transformant les directives brutes en vecteurs exploitables par le Machine Learning.
- Une évaluation de la puissance prédictive du robots.txt sur les principales catégories du web.

2 État de l'art

La classification automatique de sites web est un domaine de recherche prolifique, stimulé par l'évolution constante des technologies web. La littérature propose diverses approches que l'on peut regrouper en cinq grandes catégories. Chacune présente un compromis spécifique entre la granularité de la compréhension sémantique et le coût computationnel associé à l'extraction des caractéristiques (*feature extraction*).

La stratégie que nous proposons étant très novatrice et atypique, il est intéressant de prendre en considération les autres stratégies déjà existantes dans le domaine de la classification des sites web.

2.1 Classification basée sur le contenu (Content-Based)

Cette approche historique reste la plus largement documentée. Elle consiste à extraire le texte visible d'une page web (généralement la page d'accueil) pour y appliquer des techniques de Traitement du Langage Naturel (NLP). Les travaux fondateurs de [Joachims \(1998\)](#) ont démontré l'efficacité des Machines à Vecteurs de Support (SVM) combinées à des représentations en Bag-of-Words (BoW) ou TF-IDF pour catégoriser des documents textuels ([Qi and Davison, 2009](#)). Plus récemment, l'avènement des architectures Transformers et des modèles de langage pré-entraînés (tels que BERT) a permis de capturer le contexte sémantique complexe et polysémique des pages web ([Devlin et al., 2018](#); [Minaee et al., 2021](#)).

- **Avantages :** Une très haute précision sémantique, le texte reflétant fidèlement le sujet traité par le créateur du site.
- **Inconvénients :** Extrêmement coûteuse en bande passante. De plus, face à la prolifération des *Single Page Applications* (SPA), les scrappers statiques échouent souvent à récupérer le texte, nécessitant l'exécution préalable (et lourde) de code JavaScript.

2.2 Classification lexicale et basée sur l'URL (URL-Based)

Pour contourner la lourdeur du téléchargement HTML, de nombreux chercheurs se sont tournés vers l'analyse exclusive de l'URL ([Kan and Thi, 2005](#)). Initialement basée sur l'extraction d'heuristiques et de *n-grams* (longueur de l'URL, présence de mots-clés spécifiques), cette méthode a

fortement évolué avec le Deep Learning. Des architectures comme *URLNet* ([Le et al., 2018](#)) utilisent des Réseaux de Neurones Convolutifs (CNN) au niveau des caractères et des mots pour apprendre conjointement les représentations de l'URL sans ingénierie de caractéristiques manuelle, une méthode particulièrement performante pour détecter le phishing ([Ma et al., 2009](#)).

- **Avantages :** Une rapidité d'exécution inégalée et une consommation de bande passante quasi nulle avant même d'initier une requête HTTP complète.
- **Inconvénients :** Une forte dépendance à la clarté de l'URL. Les domaines courts, les raccourcisseurs de liens ou les noms de marque abstraits (ex: *amazon.com*) masquent le signal sémantique.

2.3 Classification basée sur le rendu visuel (Visual-Based)

Traitant les pages web comme des images, cette approche tente de reproduire la perception humaine. Les travaux pionniers de [Cai et al. \(2003\)](#) avec l'algorithme VIPS (*Vision-based Page Segmentation*) ont cherché à extraire la structure du site en mappant l'arbre DOM à des blocs visuels. Aujourd'hui, cette méthode s'appuie sur des navigateurs *headless* pour capturer l'interface, puis utilise des CNN pour analyser spatialement la mise en page (ex: une grille d'images indique souvent un site de e-commerce) ([Asirvatham and Ravi, 2001](#); [Wang et al., 2021](#)).

- **Avantages :** Totalement agnostique de la langue et particulièrement résistante aux techniques d'offuscation de code (*cloaking*). Elle capture l'expérience visuelle finale de l'utilisateur.
- **Inconvénients :** C'est l'approche la plus onéreuse. Le temps de rendu par le navigateur et le traitement d'images par GPU limitent son application à très grande échelle.

2.4 Classification basée sur les graphes (Graph/Link-Based)

Cette méthode s'appuie sur la topologie du web plutôt que sur le contenu individuel des pages. Inspirée par des algorithmes fondateurs tels que *HITS* ([Kleinberg, 1999](#)) et *PageRank* ([Page et al., 1999](#)), elle part du postulat que des sites fortement

interconnectés partagent souvent des thématiques similaires (Chakrabarti et al., 1998). Récemment, l'utilisation de Réseaux de Neurones sur Graphes (GNN) a permis d'améliorer la classification de nœuds (sites) au sein de graphes hétérogènes (Kipf and Welling, 2016).

- **Avantages** : Extrêmement robuste face à la manipulation de contenu, elle évalue la réputation et le contexte d'un site à travers son écosystème d'hyperliens.
- **Inconvénients** : Nécessite le calcul et le maintien d'une cartographie massive du web. Elle souffre également du problème de "démarrage à froid" (*cold start*) pour les nouveaux domaines.

2.5 Classification basée sur la structure technique (Structure-Based)

Au lieu de lire le texte, cette famille d'approches étudie le "squelette" du site. Des algorithmes ont par exemple utilisé l'analyse de la densité textuelle dans l'arbre DOM pour différencier le contenu principal du *boilerplate* (menus, publicités) (Kohlschütter et al., 2010), ou analysé les en-têtes HTTP (Kushmerick, 1999). C'est dans cette continuité que s'inscrit notre travail. Cependant, au lieu de parser le DOM complet ou les paquets réseau, nous proposons d'exploiter un artefact technique standardisé, déclaratif et ultra-léger : le fichier `robots.txt`. À notre connaissance, aucune étude n'a systématiquement évalué la capacité des directives destinées aux *crawlers* à agir comme un proxy d'identification des Systèmes de Gestion de Contenu (CMS), et par conséquent, de la thématique globale du domaine.

Contrairement à la classification Content-Based qui est lente et peut être bloquée par les scripts JavaScript, et à la classification URL-Based qui manque de contexte sémantique, notre approche par les `robots.txt` offre un compromis idéal : elle est presque aussi légère à récupérer qu'une URL, mais révèle une empreinte structurelle profonde du site (le CMS).

Classification	Avantages
Content-Based	Haute précision sémantique, reflet fidèle du sujet traité.
URL-Based	Vitesse d'exécution, aucune requête HTTP complète requise.
Visual-Based	Agnostique de la langue, résistant à l'offuscation (<i>cloaking</i>).
Graph/Link-Based	Évalue la réputation et le contexte via l'écosystème du web.
Structure-Based	Compromis idéal : léger à extraire, révèle la pile technologique (CMS).

Table 1: Résumé des avantages des différentes approches de classification de sites web.

Classification	Inconvénients
Content-Based	Coûteux en bande passante, vulnérable aux applications web modernes (JavaScript).
URL-Based	Signal sémantique faible ou absent (noms de marque abstraits, liens courts).
Visual-Based	Nécessite une très forte puissance de calcul (GPU, navigateurs <i>headless</i>).
Graph/Link-Based	Exige une cartographie massive du web, problème de démarrage à froid (<i>cold start</i>).
Structure-Based	Classification sémantique moins granulaire que l'analyse directe de texte.

Table 2: Résumé des inconvénients majeurs associés à chaque méthode de classification.

3 Contexte : Le Protocole d'Exclusion des Robots

Avant de détailler notre méthodologie de classification, il convient de définir formellement l'artefact au cœur de notre étude. Le Protocole d'Exclusion des Robots (REP), implémenté via le fichier `robots.txt`, est un standard de facto du web créé en 1994. Placé à la racine d'un site web (ex: `https://example.com/robots.txt`), il fournit des directives aux robots d'indexation (*crawlers*) sur les parties du site qui peuvent ou ne peuvent pas être explorées.

3.1 Anatomie et Syntaxe

Le fichier est composé d'enregistrements en texte brut, structurés de manière déclarative. Un bloc typique cible un robot d'exploration spécifique (User-agent) et liste des règles d'inclusion (Allow) ou d'exclusion (Disallow) basées sur les chemins d'URL. L'emplacement du plan du site (Sitemap) est parfois également renseigné.

```
User-agent: *
Disallow: /wp-admin/
Allow: /wp-admin/admin-ajax.php
```

```
User-agent: SemrushBot
Disallow: /
```

Figure 1: Fichier robots.txt du site web d'Unicaen

Cet exemple de fichier robots.txt indique au crawler SemrushBot qu'il n'a le droit de référencer aucune page (interdiction d'accès à la route /), alors que tous les autres User-agent ont le droit d'accéder à la route /wp-admin/admin-ajax.php, mais pas à /wp-admin/.

3.2 Le robots.txt comme signature technique

Comme l'illustre la Figure 1, les règles d'exclusion ne sont généralement pas écrites manuellement par l'administrateur du site. Elles sont générées automatiquement par le Système de Gestion de Contenu (CMS) ou le framework e-commerce utilisé.

Ainsi, un chemin comme /wp-admin/ signe la présence de WordPress (souvent utilisé pour les blogs et médias), tandis que des chemins comme /checkout/, /cart/ ou /catalog/ sont les signatures indéniables de plateformes e-commerce (comme Shopify ou Magento). C'est cette **corrélation forte entre la signature des chemins techniques et la fonction du site** qui justifie notre hypothèse de classification.

4 Méthodologie

Afin de vérifier notre hypothèse technique sur la validité des fichiers robots.txt dans le domaine de la classification de sites web, nous avons développé un pipeline complet allant de la collecte des fichiers bruts à la création d'un ensemble de données équilibré et filtré. Cette section détaille les étapes de construction de notre *dataset*.

4.1 Définition de la taxonomie

La définition des classes thématiques (la taxonomie) est une étape cruciale pour l'entraînement de notre classifieur. En effet, il nous fallait une plage de thématiques diversifiée, et qui ne risquent pas de se croiser. Par exemple, la classification que nous considérions initialement comprenait les classes "Sport" et "E-Commerce". Cependant, il est facile de trouver des sites qui correspondent à ces deux catégories simultanément, et le modèle

que nous prévoyions de créer n'allait fonctionner qu'avec une seule classe par élément.

Ainsi, sous les conseils de l'entreprise d'audit technologique "Babbar", nous avons considéré l'utilisation de la taxonomie de Google (*Google Content Taxonomy*), qui fait figure de standard industriel pour la catégorisation du web.

Cependant, cette arborescence s'est avérée inadaptée à notre approche. La taxonomie de Google est extrêmement granulaire et purement sémantique (comportant des centaines de sous-catégories très spécifiques). Et, à l'inverse, les catégories racines de ces sous-catégories semblaient trop générales pour notre utilisation. Finalement, la structure de l'arborescence de cette taxonomie était trop inconsistante pour notre utilisation, certaines catégories ayant une très grande quantité de sous-catégories et une profondeur très différentes d'autres.

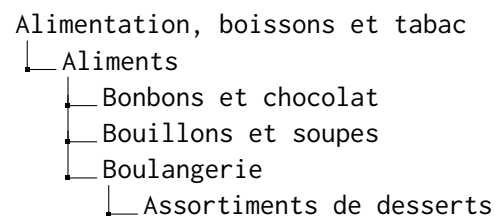


Figure 2: Extrait de la taxonomie de Google

Nous avons défini les 5 classes principales que nous allions utiliser en fonction de nos propres besoins. Ces classes étaient les suivantes :

- E-Commerce
- Actualités
- Politique
- Réseau social
- Sports

Cette classification exploratoire a finalement été écartée au profit d'une taxonomie plus adaptée, comme détaillé dans la section suivante.

4.2 Acquisition des données (Vérité Terrain)

Pour entraîner et évaluer nos modèles supervisés, nous avons besoin d'une "vérité terrain" associant de manière fiable un fichier robots.txt à la catégorie thématique de son site d'origine.

N'ayant trouvé aucun *dataset* existant qui attribue directement une catégorie à des fichiers robots.txt, nous avons essayé plusieurs méthodes pour obtenir notre propre *dataset*.

4.2.1 Récupération manuelle de sites

Notre première stratégie a été la plus simple dans la théorie, mais s’est avérée être bien trop complexe pour obtenir une grande quantité de sites.

Nous avons recherché une liste des sites les plus utilisés dans le monde par catégorie sur divers sites, et avons regroupé ces listes de sites dans notre dataset. L’inconvénient principal de cette stratégie a été que les sites les plus connus n’étaient pas forcément les plus intéressants. Egalement, ces listes de sites n’étaient pas complètes, et avaient tendance à répéter les mêmes sites entre elles. Nous avons par exemple constaté une surreprésentation systémique de plateformes majeures (ex: [x.com](https://www.x.com/), [reddit.com](https://www.reddit.com/)).

Cette stratégie nous a permis de rassembler un total de 197 sites, dans nos 5 classes équilibrées. Le faible volume de données récoltées nous a contraints à réévaluer cette stratégie.

4.2.2 Recherche de la Vérité Terrain et Datasets Existants

Notre seconde stratégie a été de rechercher des datasets existants. Comme indiqué précédemment, nous avons déjà recherché des datasets attribuant une thématique à des fichiers `robots.txt`, mais aucun dataset n’a été trouvé qui correspondait à nos attentes.

Nous avons donc changé de stratégie, et exploré plusieurs sources de données de jeux de données de référence qui attribuaient une thématique à une URL plutôt qu’à un fichier `robots.txt`:

- **Common Crawl** : Bien qu’il s’agisse du corpus web le plus exhaustif disponible publiquement (comprenant des pétaoctets d’archives et d’URL), *Common Crawl* fournit des données brutes non labellisées. L’étiquetage sémantique automatisé de ces millions d’URL (par exemple via des modèles d’IA générative ou du *zero-shot classification*) aurait nécessité des ressources de calcul et un temps de traitement hors de portée pour cette étude.
- **Annuaire web et listes de filtrage (Black-lists)** : Nous avons également étudié l’utilisation d’annuaires hiérarchiques hérités (comme DMOZ/Curlie) ou de listes de filtrage réseau maintenues par des universités (comme la *UT1 Blacklist* de l’Université Toulouse Capitole). Si ces listes offrent des classifications humaines de haute qualité, elles présentent un biais de sélection majeur : elles

sont historiquement optimisées pour restreindre l’accès à des contenus spécifiques en entreprise (pornographie, jeux d’argent, réseaux sociaux) et manquent de représentativité globale pour les sites web standard ou de niche.

Face à ces limitations — le volume inexploitable d’un côté et le biais de sélection de l’autre —, notre recherche s’est orientée vers des jeux de données préexistants, spécifiquement compilés pour des tâches de Machine Learning.

C’est au cours de cette revue que nous avons sélectionné le dataset *"Website classification using URL"* publié par Sayad Ahmed (Ahmed, 2023). Ce jeu de données présentait l’avantage décisif de fournir une liste d’URL déjà associées à une taxonomie lisible, généraliste et pertinente (E-commerce, Sports, News, etc.). Il a ainsi constitué un point de départ idéal et structuré pour initier notre propre phase de collecte ciblée des fichiers `robots.txt`. Ce dataset contient une très grande quantité de données (1.5 millions d’entrées, toutes des URLs de sites web), réparties sur un total de 15 classes variées:

Catégorie	Nombre d’éléments
Adult	35 325
Arts	253 840
Business	240 177
Computers	117 962
Games	56 477
Health	60 097
Home	28 269
Kids	46 182
News	8 989
Recreation	106 586
Reference	58 247
Science	110 286
Shopping	95 270
Society	243 943
Sports	101 328

Table 3: Distribution des effectifs par catégorie dans le jeu de données initial.

4.2.3 Manipulations sur le dataset

Afin de faire correspondre le dataset *"Website classification using URL"* à nos besoins, nous devons transformer les données de ce dataset pour obtenir les fichiers `robots.txt` des sites concernés dans les URLs.

Le principal problème de ce dataset est la présence de *duplicatas* dans les entrées. Par exemple, nous retrouvons <http://www.geocities.com/kaseychan17/index.html> et <http://www.>

geocities.com/eddie_monsoon2002/, qui sont des arborescences plus profondes de <http://www.geocities.com>. Il nous a donc fallu un premier script qui ne garde que le nom de domaine.

Un second défi majeur, lié à l'ancienneté du dataset, est l'obsolescence de nombreux liens (*link rot*). Beaucoup de domaines listés n'existent plus aujourd'hui (serveurs inaccessibles) ou ne possèdent tout simplement pas de fichier d'exclusion à leur racine, renvoyant ainsi des codes d'erreur HTTP (typiquement une erreur 404 Not Found). Nous avons donc dû intégrer à notre script de collecte une vérification stricte des codes de statut HTTP et un délai d'attente maximum (*timeout*) pour écarter ces entrées inexploitable.

Après ces manipulations, et la suppression des données qui ne correspondent pas à des fichiers robots.txt, souvent rencontrées lors de la tentative de récupération de fichier robots.txt sur des sites qui n'en possèdent pas et redirigent à la place sur une page d'erreur 404 customisée, auquel cas nous récupérons du code HTML au lieu d'un fichier robots.txt, nous nous retrouvons avec environ 300.000 données exploitables.

5 Analyse des Webcontents à travers les fichiers robots.txt

5.1 Hypothèse structurelle

Le fichier robots.txt constitue un artefact déclaratif décrivant la politique d'exploration d'un site. Contrairement aux approches basées sur le contenu, il ne contient aucune information éditoriale explicite. Notre hypothèse centrale est que la structure des règles et des chemins déclarés reflète indirectement :

- l'architecture interne du site,
- le CMS ou framework utilisé,
- la stratégie de visibilité vis-à-vis des crawlers,
- et donc, indirectement, la nature fonctionnelle du domaine.

Par exemple, des chemins tels que /wp-admin/, /checkout/, /cart/, /forum/ ou /sports/ constituent des signatures techniques fortement corrélées à certains types de sites (médias, e-commerce, forums, etc.).

Cette corrélation indirecte constitue le fondement de notre approche.

5.2 Caractéristiques extraites

Nous extrayons deux familles de caractéristiques :

1) Caractéristiques quantitatives

- nombre total de directives User-agent
- nombre de règles Disallow
- nombre de règles Allow
- nombre de Sitemap
- ratio $\frac{Disallow}{Disallow+Allow}$
- indicateur de blocage global

2) Indicateurs d'exclusion de robots Présence binaire de crawlers spécifiques : Googlebot, Bingbot, Yandex, Baiduspider, CCBot, GPTBot, AmazonBot.

Ces indicateurs visent à capturer une dimension stratégique : certains sites commerciaux bloquent par exemple des crawlers concurrents, tandis que des médias peuvent restreindre l'accès à certains robots d'archivage.

5.3 Analyse exploratoire

Une analyse descriptive préliminaire montre :

- une forte variance du nombre de règles selon les catégories ;
- une concentration élevée de motifs e-commerce dans la classe Shopping ;
- une fréquence accrue de sections /news/ ou /article/ dans la classe News ;
- une présence plus marquée de règles ciblant des bots spécifiques dans certaines catégories commerciales.

Cependant, ces corrélations restent faibles et bruitées, suggérant que le signal structurel est partiellement informatif mais non déterministe.

6 Évaluation automatique par apprentissage automatique

6.1 Jeux de données

Nous utilisons deux ensembles distincts afin d'analyser l'impact du nombre de classes et de la taille du corpus.

Le dataset_2500 permet une analyse multi-classe fine (15 catégories), tandis que le dataset_15000 permet d'évaluer la robustesse du modèle sur un volume plus important avec une taxonomie réduite.

Dataset	Instances	Classes	Distribution
dataset_2500	37 500	15	équilibrée
dataset_15000	120 000	8	équilibrée

Table 4: Caractéristiques des jeux de données. Chaque instance correspond à un fichier `robots.txt`.

6.2 Représentation vectorielle

Le contenu brut est transformé via TF-IDF sur n-grams de caractères :

```
analyzer=char, ngram_range=(3,4)
```

Cette représentation est particulièrement adaptée aux :

- motifs d'URL,
- noms de dossiers,
- signatures techniques,
- noms de crawlers.

Contrairement aux word n-grams, les character n-grams capturent les structures partielles et sont robustes aux variations mineures.

6.3 Modèles évalués

Nous comparons :

1. Régression Logistique (baseline linéaire)
2. Random Forest (features structurelles + bots)
3. LinearSVC (marge maximale)

6.4 Résultats expérimentaux

Dataset	Modèle	Accuracy	Macro-F1
2500 (15c)	LR TF-IDF (4k)	0.2101	0.2032
2500 (15c)	RF (1.5k + bots)	0.1844	0.1779
2500 (15c)	LinearSVC (8k)	0.2831	0.2862
15000 (8c)	LR TF-IDF (4k)	0.3089	0.3009

Table 5: Performances sur un split stratifié 80/20. Les scores à 8 classes ne sont pas directement comparables à ceux à 15 classes.

6.5 Interprétation

Plusieurs observations émergent :

- Les modèles linéaires surpassent la Random Forest sur données sparse.
- Les features liées aux exclusions de bots apportent peu d'amélioration mesurable.

- La réduction du nombre de classes améliore mécaniquement l'accuracy.

- Malgré l'absence de contenu sémantique explicite, le signal structurel permet une performance nettement supérieure au hasard (6.7% pour 15 classes).

Ces résultats confirment que le `robots.txt` contient une information exploitable, mais insuffisante pour rivaliser avec des approches content-based.

7 Développement d'un classifieur ad-hoc

7.1 Sélection du modèle final

Le modèle retenu maximise la Macro-F1 sur le `dataset_2500` :

Composant	Configuration
Vectorisation	TF-IDF caractères
n-grams	(3,4)
max_features	8000
min_df	2
sublinear_tf	True
Classifieur	LinearSVC
C	2.0
class_weight	balanced

Table 6: Configuration du modèle final déployé.

7.2 Pipeline d'inférence

Le pipeline ad-hoc comprend :

1. récupération du `robots.txt` ;
2. vectorisation via le TF-IDF entraîné ;
3. prédiction via SVM linéaire ;
4. classement top-k via la fonction de décision.

7.3 Cas d'usage

Exemple : `https://lemonde.fr` est correctement classé dans la catégorie **News**.

Une interface Streamlit permet :

- la prédiction par URL ;
- la prédiction par texte brut ;
- l'affichage des top-k classes.

7.4 Discussion finale

Le classifieur ad-hoc démontre qu'un signal purement structurel peut capturer une information thématique non triviale.

Néanmoins, la performance reste modérée comparée aux approches content-based, soulignant que le robots.txt constitue un proxy partiel et bruité de la nature réelle du site.

References

- Sayad Ahmed. 2023. [Website classification using url](#). Dataset accessible en ligne.
- Arul Prakash Asirvatham and Karthik Ravi. 2001. Web page classification based on visual features. In *Proceedings of the 2001 international conference on World Wide Web*, pages 1–6.
- Deng Cai, Shipeng Yu, Ji-Rong Wen, and Wei-Ying Ma. 2003. Vips: a vision-based page segmentation algorithm. In *Microsoft Research Technical Report*.
- Soumen Chakrabarti, Byron Dom, and Piotr Indyk. 1998. Judging with and by the company you keep. In *Proceedings of the 1998 ACM SIGKDD*, pages 1–6.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.
- Thorsten Joachims. 1998. Text categorization with support vector machines: Learning with many relevant features. In *European conference on machine learning*, pages 137–142. Springer.
- Min-Yen Kan and Hoang Oanh Nguyen Thi. 2005. Fast webpage classification using url features. In *Proceedings of the 14th ACM international conference on Information and knowledge management*, pages 325–326.
- Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*.
- Jon M Kleinberg. 1999. Authoritative sources in a hyperlinked environment. *Journal of the ACM (JACM)*, 46(5):604–632.
- Christian Kohlschütter, Peter Fankhauser, and Wolfgang Nejdl. 2010. Boilerplate detection using shallow text features. In *Proceedings of the third ACM international conference on Web search and data mining*, pages 441–450.
- Nicholas Kushmerick. 1999. Learning to remove internet advertisements. *Proceedings of the third annual conference on Autonomous Agents*, pages 175–181.
- Hung Le, Quang Pham, Doyen Sahoo, and Steven CH Hoi. 2018. Urlnet: Learning a url representation with deep learning for malicious url detection. *arXiv preprint arXiv:1802.03162*.
- Justin Ma, Lawrence K Saul, Stefan Savage, and Geoffrey M Voelker. 2009. Beyond blacklists: learning to detect malicious web sites from suspicious urls. In *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 1245–1254.
- Shervin Minaee, Nal Kalchbrenner, Erik Cambria, Narjes Nikzad, Meysam Chenaghlu, and Jianfeng Gao. 2021. Deep learning-based text classification: a comprehensive review. *ACM computing surveys (CSUR)*, 54(3):1–40.
- Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. The pagerank citation ranking: Bringing order to the web. *Stanford InfoLab*.
- Xiaoguang Qi and Brian D Davison. 2009. Web page classification: Features and algorithms. *ACM computing surveys (CSUR)*, 41(2):1–31.
- Yuting Wang and 1 others. 2021. A visual-based approach for web page classification using deep learning. *IEEE Access*, 9:45123–45135.